def battle(prompt):
 msgs = [{"role": "user", "content": prompt}]

```
None
# Model A — OpenAI's GPT
a = openai_client.chat.completions.create(model="gpt-4o-mini",
messages=msgs)

# Model B — Llama on Groq (same code, different brain!)
b =
groq_client.chat.completions.create(model="llama-3.3-70b-versatile"
, messages=msgs)

return a.choices[0].message.content, b.choices[0].message.content
```

…now we wrap this in Gradio with two columns + thumbs up/down 👇

explain the code of python one by one

Here's a line-by-line walkthrough.

1. Function definition

```
Python
def battle(prompt):
```

Defines a function named `battle` that takes one input, `prompt` (the user's question as a string).

2. Build the message list

```python
msgs = [{"role": "user", "content": prompt}]
```

Chat APIs expect a **list of messages**, each a dict with:

- `role`: who is speaking (`"user"`, `"assistant"`, or `"system"`)
- `content`: the actual text

Here there is one message, from the user. Both models receive this *same* list, which keeps the comparison fair.

3. Call Model A (OpenAI)

```python
a = openai_client.chat.completions.create(model="gpt-4o-mini", messages=msgs)
```

- `openai_client` is an already-created OpenAI client object (e.g. `OpenAI(api_key=...)`), defined elsewhere.
- `.chat.completions.create(...)` sends the chat request.
- `model="gpt-4o-mini"` picks the model.
- `messages=msgs` passes the conversation.
- The result `a` is a response object, not plain text.

4. Call Model B (Llama on Groq)

```python
b = groq_client.chat.completions.create(model="llama-3.3-70b-versatile", messages=msgs)
```

Same structure, different provider. Groq hosts open-source models such as Llama and deliberately mirrors OpenAI's API shape, so the call looks nearly identical. Only the client and model name change. That is what the comment "same code, different brain" means.

5. Extract and return the text

```python
return a.choices[0].message.content, b.choices[0].message.content
```

- `a.choices` is a list of possible completions (usually just one).
- `[0]` takes the first.
- `.message.content` is the generated text.
- Returning two values gives a **tuple** `(answer_from_A, answer_from_B)`.

6. The Gradio part (the final comment)

Gradio would wrap `battle` in a UI:

- **Two output columns**, one per model, which is why the function returns two strings.
- **Thumbs up/down buttons** so the user can vote on which answer is better, turning it into a blind-style model comparison.

pip install openai gradio python-dotenv

```
import os
 import gradio as gr
 from dotenv import load_dotenv
 from openai import OpenAI
 load_dotenv()
```

OpenAI client

```
openai_client = OpenAI(
 api_key=os.getenv("OPENAI_API_KEY")
 )
 # uses OPENAI_API_KEY
 groq_client = OpenAI(api_key=os.getenv("GROQ_API_KEY"),
 base_url="https://api.groq.com/openai/v1")
```

```
def ask(client, model, prompt):
 r = client.chat.completions.create(
 model=model, messages=[{"role": "user", "content": prompt}])
 return r.choices[0].message.content

def battle(prompt):
 a = ask(openai_client, "gpt-4o-mini", prompt)
 b = ask(groq_client, "openai/gpt-oss-120b", prompt)
 return a, b

def vote(label):
 return f"🗳️ Thanks! You voted: {label}" # in real apps, save this to a file/DB

with gr.Blocks(title="LLM Arena") as demo:
 gr.Markdown("# 🥊 LLM Arena — one prompt, two models")
 prompt = gr.Textbox(label="Ask both models the same thing")
 go = gr.Button("⚔️ Battle!", variant="primary")
```

None

```
with gr.Row():

    with gr.Column():

        gr.Markdown("### 🤖 Model A")

        out_a = gr.Markdown()

        with gr.Row():

            up_a   = gr.Button("👍");  down_a = gr.Button("👎")

    with gr.Column():

        gr.Markdown("### 🤖 Model B")

        out_b = gr.Markdown()

        with gr.Row():

            up_b   = gr.Button("👍");  down_b = gr.Button("👎")


verdict = gr.Markdown()
```

```
go.click(battle, inputs=prompt, outputs=[out_a, out_b])

up_a.click(lambda: vote("👍 Model A"), outputs=verdict)

down_a.click(lambda: vote("👎 Model A"), outputs=verdict)

up_b.click(lambda: vote("👍 Model B"), outputs=verdict)

down_b.click(lambda: vote("👎 Model B"), outputs=verdict)
```

demo.launch(share=True) # → local + public link 🎉
explain the code line by line

1. Imports and setup

Python

```
# pip install openai gradio python-dotenv
```

A comment listing the packages to install.

Python

```
import os

import gradio as gr

from dotenv import load_dotenv

from openai import OpenAI

load_dotenv()
```

- `os`: reads environment variables.
- `gradio as gr`: the library that builds the web UI.

- `load_dotenv`: loads key/value pairs from a `.env` file into environment variables.
- `OpenAI`: the client class. It is used for **both** providers here (see below).
- `load_dotenv()` actually runs the loading. Your `.env` should contain `OPENAI_API_KEY=...` and `GROQ_API_KEY=...`.

2. Creating the clients

```python
openai_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
```

Creates the client for OpenAI. `os.getenv` reads the key from the environment, so it is never hardcoded.

```python
groq_client = OpenAI(api_key=os.getenv("GROQ_API_KEY"),
                     base_url="https://api.groq.com/openai/v1")
```

This is the key trick. Groq's API is OpenAI-compatible, so you reuse the same `OpenAI` class and point `base_url` at Groq's server. That is why you only need one SDK.

3. Helper to call any model

```python
def ask(client, model, prompt):
    r = client.chat.completions.create(
        model=model, messages=[{"role": "user", "content":
prompt}])
    return r.choices[0].message.content
```

- Takes a client, a model name, and the prompt.
- Wraps the prompt in the chat format (`role` + `content`).
- Returns just the text from the first choice.

Because every client has the same interface, one function serves both providers.

4. The battle function

```python
def battle(prompt):
    a = ask(openai_client, "gpt-4o-mini", prompt)
    b = ask(groq_client, "openai/gpt-oss-120b", prompt)
    return a, b
```

Sends the same prompt to both models (GPT-4o mini on OpenAI, `gpt-oss-120b` hosted on Groq) and returns two strings. Gradio maps those two return values onto two output components.

5. The vote function

```python
def vote(label):
    return f"🗳️ Thanks! You voted: **{label}**"
```

Returns a confirmation message in Markdown. It does not store anything. As the comment says, a real app would save the vote to a file or database.

6. Building the UI

```python
with gr.Blocks(title="LLM Arena") as demo:
```

`Blocks` is Gradio's flexible layout API. Everything indented inside is part of the page. `title` sets the browser tab title.

```python
gr.Markdown("# 🥊 LLM Arena — one prompt, two models")

prompt = gr.Textbox(label="Ask both models the same thing")
```

```
go = gr.Button("⚔️ Battle!", variant="primary")
```

- A heading.
- A text box for the user's prompt.
- A highlighted button (`variant="primary"`) that triggers the battle.

Python
```
with gr.Row():
    with gr.Column():
        gr.Markdown("### 🤖 Model A")
        out_a = gr.Markdown()
        with gr.Row():
            up_a = gr.Button("👍");  down_a = gr.Button("👎")
    with gr.Column():
        gr.Markdown("### 🤖 Model B")
        out_b = gr.Markdown()
        with gr.Row():
            up_b = gr.Button("👍");  down_b = gr.Button("👎")
```

- `Row` lays children side by side. Each `Column` is one model's panel.
- `out_a` and `out_b` are empty Markdown components that will display each answer (Markdown renders code blocks and lists nicely).
- Each column has its own thumbs up/down buttons in a nested `Row`.
- The `;` just puts two statements on one line.

Python
```
verdict = gr.Markdown()
```

An empty area that shows the vote confirmation.

7. Wiring events

```python
go.click(battle, inputs=prompt, outputs=[out_a, out_b])
```

When the button is clicked, Gradio reads the textbox value, passes it to `battle`, and puts the two returned strings into `out_a` and `out_b`, in order.

```python
up_a.click(lambda: vote("👍 Model A"), outputs=verdict)

down_a.click(lambda: vote("👎 Model A"), outputs=verdict)

up_b.click(lambda: vote("👍 Model B"), outputs=verdict)

down_b.click(lambda: vote("👎 Model B"), outputs=verdict)
```

Each button calls `vote` with a fixed label via a `lambda` (no inputs needed) and shows the result in `verdict`.

8. Launching

```python
demo.launch(share=True)
```

Starts a local server and, with `share=True`, also creates a temporary public `gradio.live` URL. Anyone with the link can use your app **and spend your API credits**, so use it carefully.

Interview-ready limitations

- **Sequential calls:** `a` finishes before `b` starts, so the user waits for both. Use threads or async.
- **No error handling:** if either API fails, the UI shows an error and neither answer appears.
- **Votes aren't stored:** and the buttons stay active after voting, so users can vote repeatedly.
- **Not blind:** the models are labeled A/B in a fixed order. A fair arena would randomize positions and hide model names until after the vote.
- **Public link risk:** `share=True` exposes the app and your keys' quota.

I can rewrite this with parallel calls, error handling, randomized blind positions, and votes saved to a CSV if you like.